Kafka Consume & Produce: At-Least-Once Delivery

Rob Golder - August 2022

Introduction

Kafka messaging provides a guarantee of at-least-once delivery by default. This article looks at what this means, how duplicate message delivery can happen, and what the implications are for the application. It suggests further reading with approaches and patterns that can be adopted to deal with, or stop, duplicate delivery.

At-Least-Once Delivery

At-least-once delivery means that a message (in fact a batch of messages) is guaranteed to be delivered at least once, but could be delivered multiple times in failure scenarios.

The companion Spring Boot application that has at-least-once delivery is detailed in the Kafka Consumer & Produce: Spring Boot Demo article. The full source code is available here.

Consumer Offsets

In the consume and produce example from the Spring Boot demo application for each message that is consumed an outbound message is written to the outbound topic. However there is a further write (or writes) that happens upon completion of the outbound topic writes for the consumed batch. In order to mark the messages as consumed, the internal Kafka consumer offsets topic is written to with the offsets of the successfully consumed messages. There will be one write to the consumer offsets topic for every partition from which messages within the batch were consumed. This is done automatically by default by the Kafka client library, as is the case in the demo application, as the enable.auto.commit consumer configuration parameter is set to true.

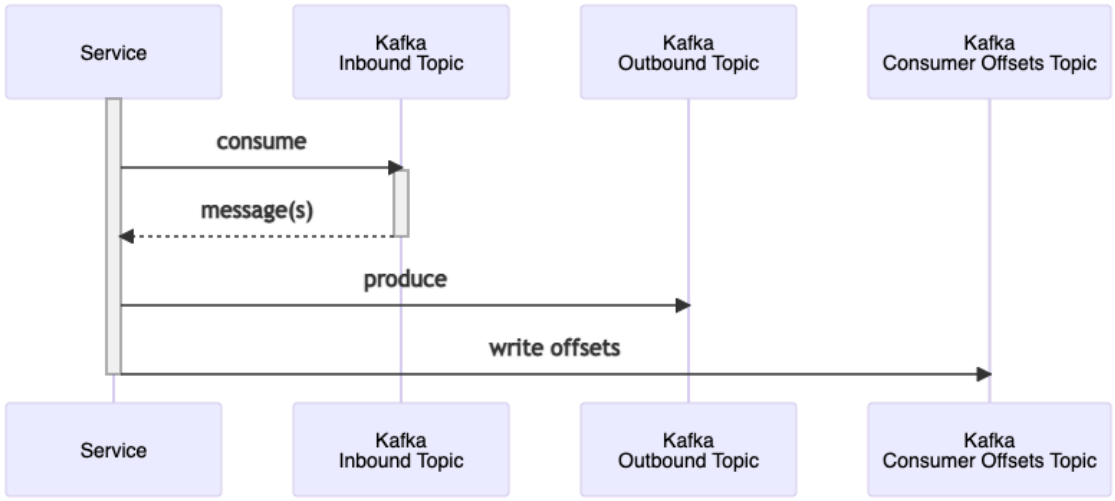

Figure 1: Consume, produce, write offsets

Following the write to the consumer offsets topic the consumer is ready to poll for the next batch of messages.

Failure Scenarios

If there is some failure during the processing, such as the service instance dies, or perhaps takes too long to process the messages and times out, then the messages are not marked as consumed. This means when the consumer group is rebalanced and the topic partition is assigned to a new consumer instance, or indeed to the same consumer instance when it has restarted, then on the next poll the same batch of messages are redelivered.

Service Failure

The following diagram shows two service instances, each with a consumer in the same consumer group listening to the inbound topic. A failure occurs after the outbound messages have been produced, but before the write to the consumer offsets topic. The consumer group rebalances, the one remaining consumer rejoins the group, and is assigned the topic partition. This consumer polls the partition and receives the same, duplicate, batch of messages. All processing that had completed on the initial consume occurs again, resulting here in a duplicate produce to the Kafka outbound topic.

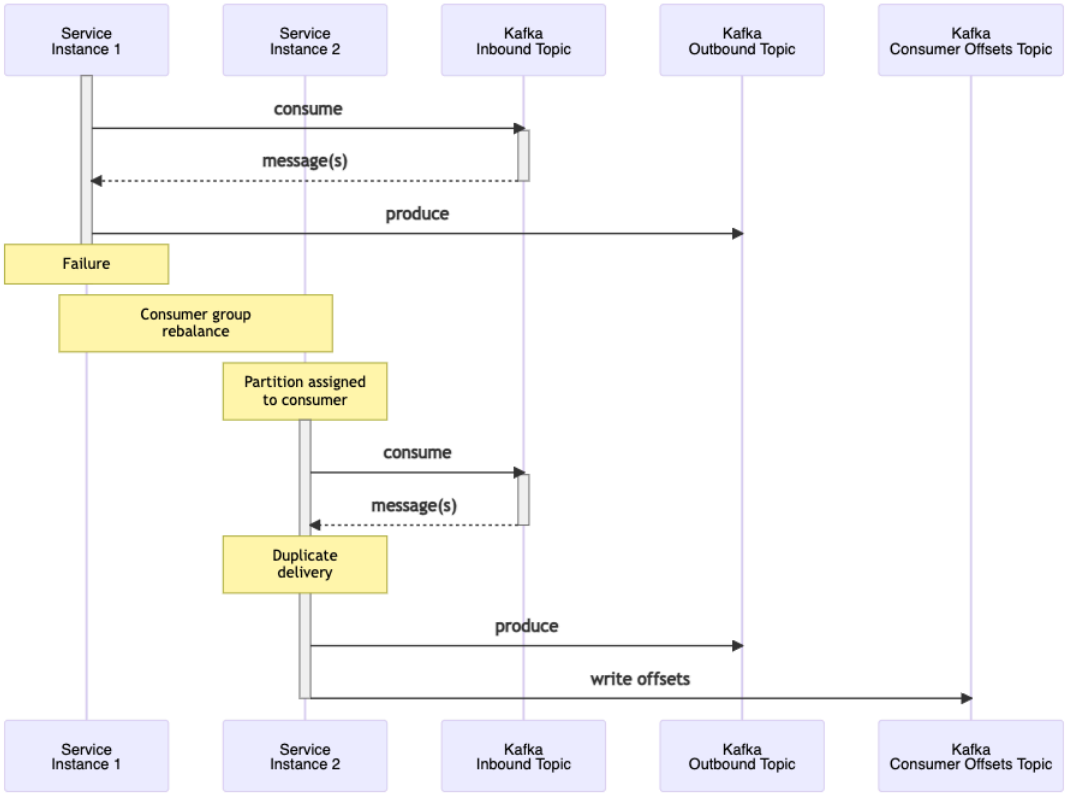

Figure 2: Service failure - duplicate delivery

Poll Timeout

The same duplicate delivery can occur under a different scenario, when the consume poll times out. Processing of each batch of messages consumed on a poll must complete within the max.poll.interval.ms. If this does not complete in time then the broker has to consider that the consumer has died, and therefore trigger a consumer group rebalance. If it did not do this, and the consumer had indeed died as in the example above, the topic partition would be blocked as it would have no active consumer.

In the example below, the max poll timeout is exceeded after the produce of messages to the outbound topic, but before the consumer offsets are written to. While this is a very small window within which the timeout could occur such a scenario can and does happen, and should be catered for. As above, the consumer group is rebalanced, and the topic partition is assigned to the other consumer in the consumer group, in this case service instance 2. This instance then consumes the duplicate batch of messages.

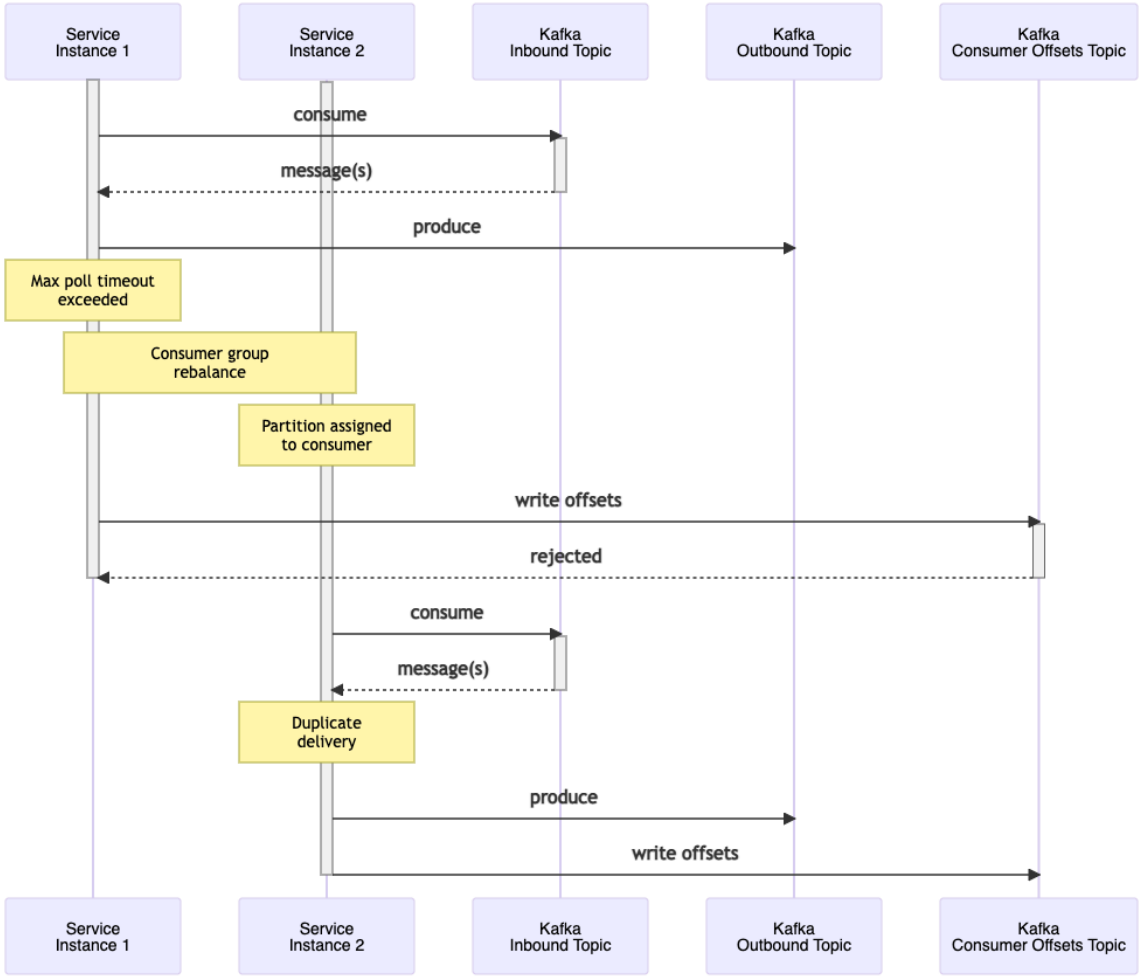

Figure 3: Poll timeout - duplicate delivery

Meanwhile the first consumer is still active, and attempts to write the consumer offsets to mark the batch as consumed as its final action before completing its poll. As the consumer is no longer assigned the topic partition for which it is writing the offsets, this write is rejected. Now that its poll is complete it informs the broker that it is again active by sending a request to rejoin the consumer group. This triggers another consumer group rebalance, and the topic partitions are reassigned between the consumer in the consumer group.

Single Service Instance Failure

In both of the failure scenarios drawn out above there are two consumer instances in the consumer group. Where there is only a single service, with a single consumer, whether the service fails and restarts, or the poll times out, the result is the same. Once the consumer is restarted, or the poll that timed out has completed, this consumer will rejoin the consumer group, and as the sole member it will receive the same batch of messages again.

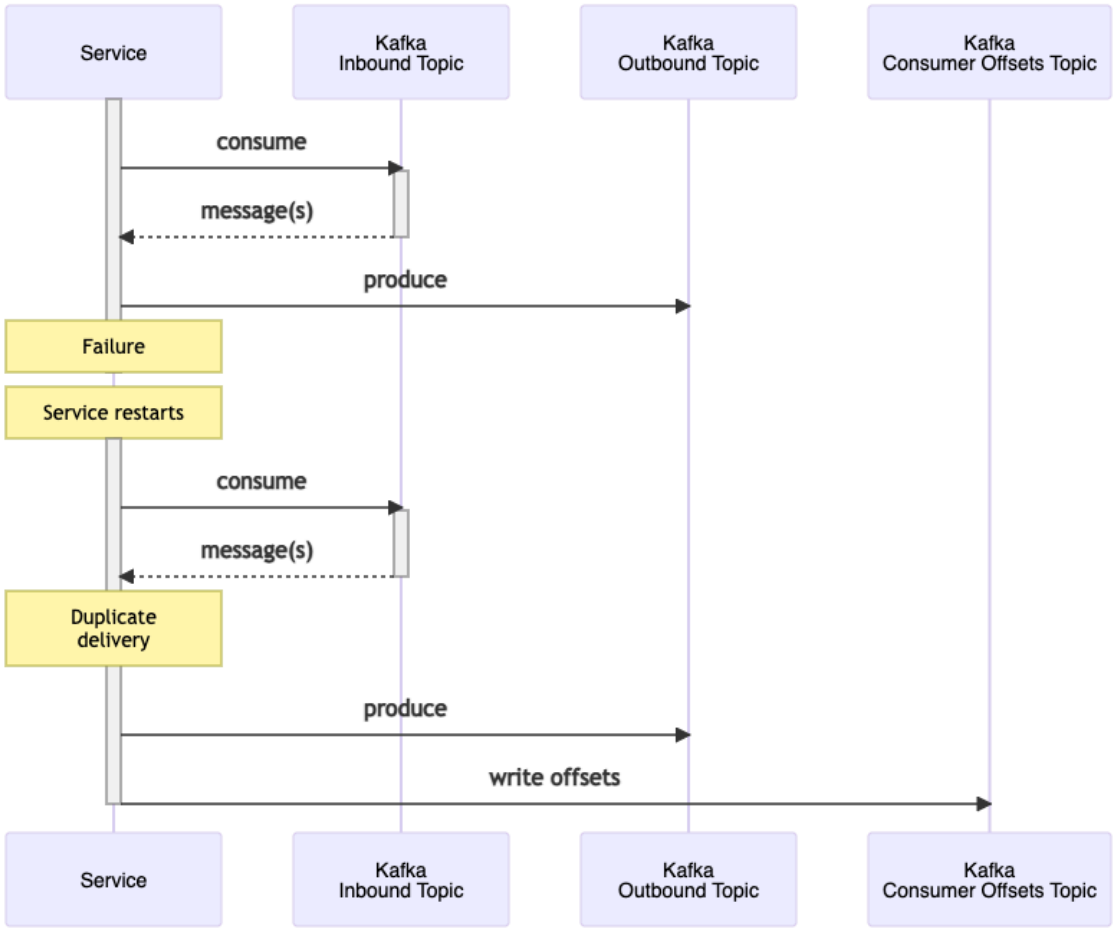

Figure 4: Service failure - single consumer - duplicate delivery

Consumer Group Rebalance

Consumer group rebalances can be costly, as typically no further messages are consumed while the partitions are rebalancing. However there are means to reduce the impact via configuring the rebalance behaviour, which is covered in the Kafka Consumer Group Rebalance article.

Duplicate Processing

The diagrams in the preceding sections illustrate a simple application that consumes a batch of messages and produces resulting outbound messages. Typically of course message processing will encompass many other actions, such as writing to the database and making HTTP calls to third party services. The failure scenarios described could therefore result in duplicate database writes, and duplicate HTTP calls.

At-least-once delivery, with duplicate processing as the outcome, may be an acceptable outcome when using Kafka as the messaging broker. If not, there are many different options and ways to address this. These are covered in detail in other articles, including applying different deduplication patterns, such as using the Idempotent Consumer pattern.

Alternative Processing Guarantees

By default Kafka guarantees at-least-once message processing. Kafka can also be configured to guarantee at-most-once message processing, or to guarantee exactly-once messaging semantics.

At-Most-Once

At-most-once processing is achieved on the consumer side by immediately marking as consumed the batch of messages before any further processing has occurred. This ensures that if they are redelivered for some reason (such as in the failure scenarios covered above) that no duplicate processing will occur. However if the processing fails before completion, the messages will not be redelivered, so those messages are essentially lost. To mark the messages as consumed the enable.auto.commit consumer configuration parameter is set to false, and the developer is responsible for ensuring the writes to the consumer offsets topic happens prior to the message processing.

Exactly-Once

Exactly-once messaging semantics can be achieved by using Kafka Transactions. While reasonably trivial to configure, the semantics and the ramifications of enabling transactions is a large topic, and covered in detail in the Kafka Transactions article.

Conclusion

Out of the box Kafka provides at-least-once messaging. For many use cases this will suffice, but it is important to understand the failure cases where duplicate messages can occur, and whether the resulting duplicate processing is acceptable. If duplicates cannot be tolerated then there are different deduplication patterns and approaches that can be employed to cater for them or avoid them altogether. These include deduplicating with the Idempotent Consumer pattern, and configuring exactly-once messaging semantics with Kafka Transactions.

Source Code

The source code for the accompanying Spring Boot demo application is available here:

https://github.com/lydtechconsulting/kafka-springboot-consume-produce/tree/v1.0.0

More On Kafka: Consume & Produce

The following accompanying articles cover the Consume & Produce flow:

- Kafka Consume & Produce: Spring Boot Demo: details the companion application.
- Kafka Consume & Produce: Testing: covers the tools and frameworks available to comprehensively test a Spring Boot application using Kafka as the messaging broker.